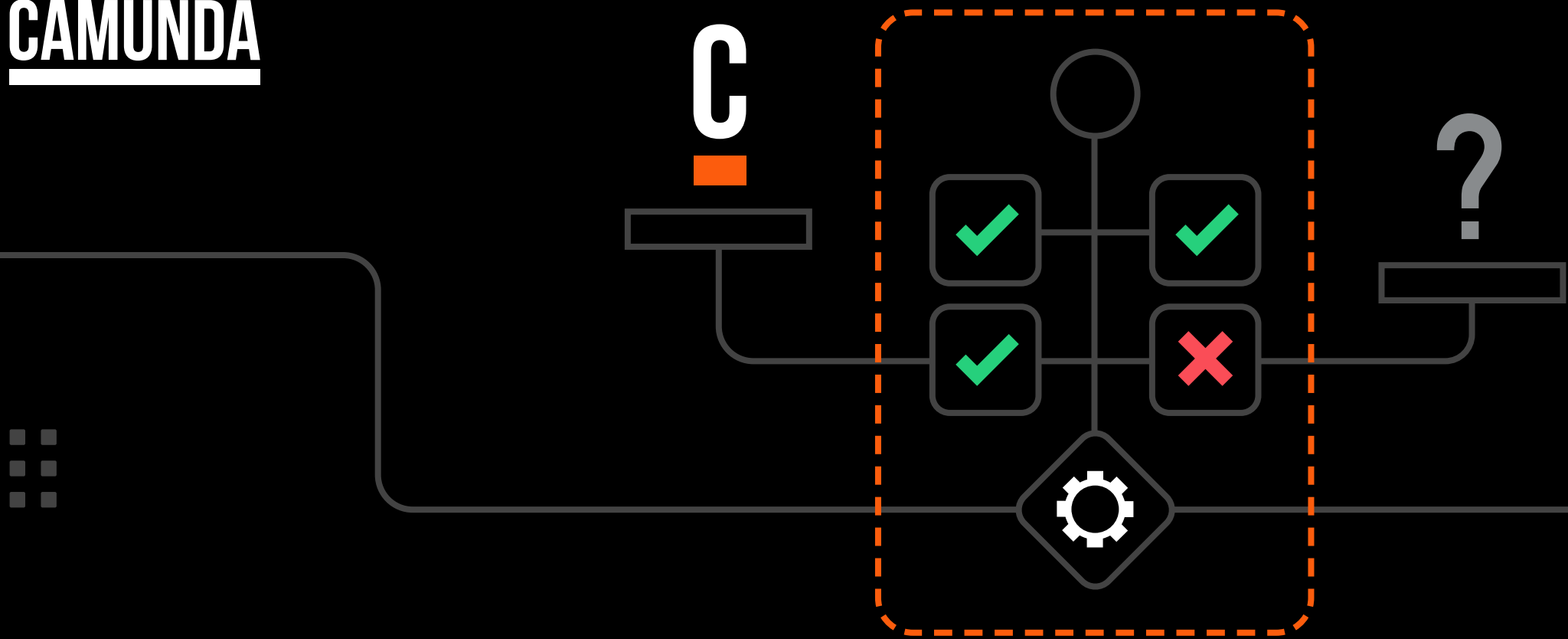




Camunda Compared to Alternatives

Contents

Executive Summary	3
Every Business Process is Now Legacy	4
Understanding Deterministic, Dynamic, and Agentic Orchestration	5
The Orchestration and Automation Landscape	6
Process Automation Tools	9
Monolithic Automation Platforms	11
Microservices Orchestrators	13
Task and Integration Automation Tools	15
AI Agent Frameworks and Platforms	17
Agentic Orchestration with Camunda	19
Conclusion	23

Executive Summary

AI is getting more capable every quarter, and enterprises are deploying AI agents faster than ever. Yet there is a widening gap between what AI can do and what enterprises actually get from it. End-to-end business processes still depend on manual handoffs, human exception handling, and stitched-together integrations. Each new bot or agent adds capability but also fragmentation. Results flatten, costs rise, and the organization hits an automation ceiling.

The journey from pilot to production is longer and harder than expected. The reason isn't that agents can't do the work; it's that coordinating agents, people, and systems safely at scale requires something many organizations don't yet have: orchestration.

This whitepaper analyzes the categories of software tools organizations use to automate their business processes. Each delivers real value, but none, on its own, can orchestrate a complex business process from end to end or safely give AI agents real operational authority. That requires agentic orchestration: blending deterministic process logic (every step predefined) with dynamic, AI-driven decision-making (agents determine the next best action) in one process model, executed and governed by one workflow engine.

Camunda is the enterprise platform for agentic orchestration, orchestrating AI agents, people, and systems across end-to-end business processes and uniquely blending deterministic and dynamic orchestration. Through its composable architecture, Camunda meets enterprises where they are and grows with them as AI evolves. The result: AI-powered automation that actually delivers at scale, with the trust, governance, and control that mission-critical operations demand.



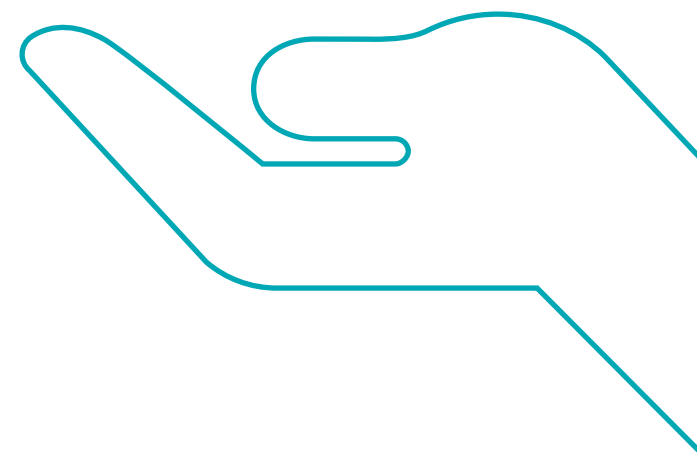
Every Business Process is Now Legacy

Here's an uncomfortable idea worth confronting before comparing software tools: every process currently running your business is legacy. Legacy doesn't mean old or poorly designed; a process becomes legacy when the assumptions it was designed around stop being true, and every process in operation today was designed for a world in which AI did not exist.

Consider what those designs encode: sequential approval chains exist because human reading time is scarce, batch processing exists because real-time analysis wasn't feasible, escalation queues exist because judgment couldn't be automated. None were design mistakes; they were rational responses to constraints that have now expired.

Legacy doesn't mean everything should be discarded, though. Some constraints are incidental technology workarounds that can fall away; others are essential, like the regulatory approval gate, separation of duties, and human accountability for a binding decision. These are the encoded judgment of the business and often the law; a serious approach to automation preserves and enforces them even when autonomous agents do the work. Nor must everything be redesigned tomorrow: for most organizations, the practical path starts with orchestrating what exists and improving it incrementally, building the foundation that makes deeper redesign possible.

AI raises the stakes on getting that foundation right. Agents that enter already-fragmented processes don't solve the fragmentation; they add another silo, at speed. With regulators demanding auditability and boards demanding measurable ROI from AI, the automation ceiling is an immediate priority, not a future consideration.



Understanding Deterministic, Dynamic, and Agentic Orchestration

Most process orchestration is deterministic: it uses predefined logic to execute a process. Dynamic orchestration (also known as non-deterministic orchestration) leverages AI agents to determine the next steps to take as a process executes. Each approach has its strengths and its downsides:

- **Deterministic orchestration:** Predictable, auditable, and ideal for structured processes with clearly defined steps and outcomes. The downside is rigidity: every path must be anticipated at design time, so exceptions pile up in queues, waiting for a person.
- **Dynamic orchestration:** Flexible and adaptive. Agents respond to real-time context and handle cases no one predicted. The downsides: results are harder to audit and less predictable, and every execution carries a compute cost that climbs quickly at enterprise volume.

Agentic orchestration blends the two, allowing deterministic logic to govern known, repeatable paths while delegating unpredictable paths to AI agents embedded within the process.

Consider Maya, a first-time homebuyer applying for a mortgage. Deterministic logic runs her credit check, verifies her identity, and enforces the regulatory steps every application requires. When her income documents arrive inconsistent (she's self-employed, with three revenue streams), an agent embedded in the process investigates, gathers context, and proposes a resolution instead of dropping her application into an exception queue. The process stays governed; Maya stays out of limbo.

Agentic orchestration is enabling automation where it wasn't possible before, such as complex case management. But AI agents are a new technology and regulations are evolving, so it's important to select a solution that enforces guardrails around AI agents and enables teams to monitor and audit them within the context of business processes.



The Orchestration and Automation Landscape

The market for automation tools is converging. Vendors with backgrounds in RPA, low-code application development (LCAP), integration, and enterprise applications have added process and agent capabilities, while process-first vendors have added AI.

Analysts have recognized this convergence: Gartner defines Business Orchestration and Automation Technologies (BOAT), where the “O” signals that orchestration, not just automation, is critical for end-to-end processes; and Forrester has identified Adaptive Process Orchestration (APO) for platforms that combine deterministic and AI-driven control flows.

Convergence, however, doesn’t erase architectural differences: where a product started still determines what it can do today.



What matters when choosing an orchestration solution

Many automation tools partially or fully deliver the following capabilities, but a robust process orchestration solution will check all of the boxes.

- **End-to-end process orchestration:** Orchestrates every automated, manual, and agent-driven task in an end-to-end business process, across endpoints such as APIs, microservices, robotic process automation (RPA) bots, Internet of Things (IoT) devices, and AI agents.
- **Agentic orchestration:** Combines deterministic and dynamic orchestration in the same process, orchestrating around AI agents and inside them to enforce required steps between an agent's decisions and its actions.
- **Support for long-running processes:** Manages processes that run for hours, days, weeks, or longer, including durable coordination between AI agents that must wait on one another, on systems, or on people.
- **Observability, auditability, and optimization:** Provides real-time and historical insight into every process and agent decision, with a complete audit trail that satisfies regulators and features that help teams manage LLM costs, identify performance bottlenecks, and continuously improve.
- **One shared process model:** Uses one process model across design, execution, monitoring, and improvement, so that business stakeholders, low-code developers, and professional developers collaborate on the same visualization throughout the lifecycle.
- **Open and composable architecture:** Components work together seamlessly and can fit into any tech stack, using any programming language. The solution can run on premises, in the cloud, or hybrid.
- **Highly scalable and resilient:** Leverages cloud-native technology to scale process volumes and provide resilience against unplanned downtime and data loss.
- **Low total cost of ownership:** Structural qualities keep long-run costs down: open standards replace a proprietary skills tax, composability replaces consulting dependency, and portable process logic survives changes in AI models, vendors, and infrastructure without rework.



The Automation Landscape

The table below summarizes how well each category satisfies the qualities required to orchestrate complex processes and AI agents from end to end.

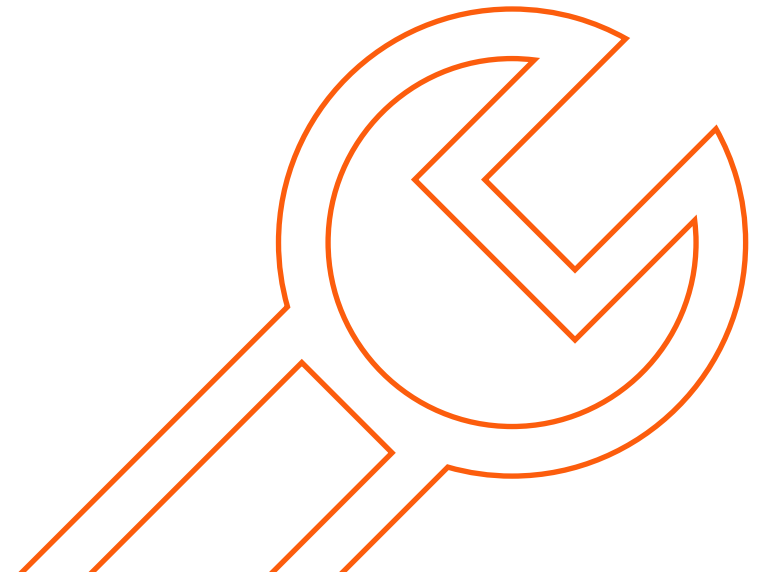
	Process automation tools	Monolithic automation platforms	Microservices orchestrators	Task and integration automation tools	AI agent frameworks and platforms	Camunda
End-to-end process orchestration	●	●				●
Agentic orchestration	●	●			●	●
Support for long-running processes	●	●	●	●	●	●
Observability, auditability, and optimization	●	●	●		●	●
One shared process model	●	●	●	●		●
Open and composable architecture	●		●	●	●	●
Highly scalable and resilient			●			●
Low total cost of ownership			●	●	●	●

- Full support
- Partial support



Process Automation Tools

When looking to increase automation across important business processes, most organizations adopt a process automation tool. These tools originated in business process management (BPM), which began as a way to model and execute straightforward, repeatable back-office processes such as invoice processing; today they handle processes at all levels of complexity, from customer onboarding to loan origination. They excel at long-running processes that combine automated tasks with human tasks, give operations teams features for troubleshooting problems in production, and provide extensive analytics because they manage execution from start to finish.



Challenges

Disconnected agentic orchestration

Many process automation tools integrate with AI services and even incorporate AI agents into processes. However, AI is usually treated as a single task or decision point within an otherwise deterministic process. There is no true dynamic orchestration (no way for an agent to determine its sequence of actions based on context, goals, and available tools), and nothing governs what happens between an agent's decision and its action. The agent is a black box: the tool invokes it and receives a result, with no visibility into the steps in between.

Unlocking the full potential of AI agents requires native, first-class integration: agents modeled in the same design environment as deterministic processes, executed by the same workflow engine, and visible at every stage. Without this, teams can't monitor agent behavior or maintain governance, which deploying agentic orchestration at scale requires.

Inflexible architecture and integration

Process automation tools need robust integration to trigger the task automation tools that execute process work, and they must accommodate custom code, which complex processes almost always require. Many fall short here. Connectors are useful until they need to be customized; if a connector is a black box, teams wait on the vendor. And if custom code must be written in a specific programming, scripting, or proprietary language, developers can't use their team's established standards, leading to skills gaps, outsourced work, and slower delivery.

Not highly scalable or resilient

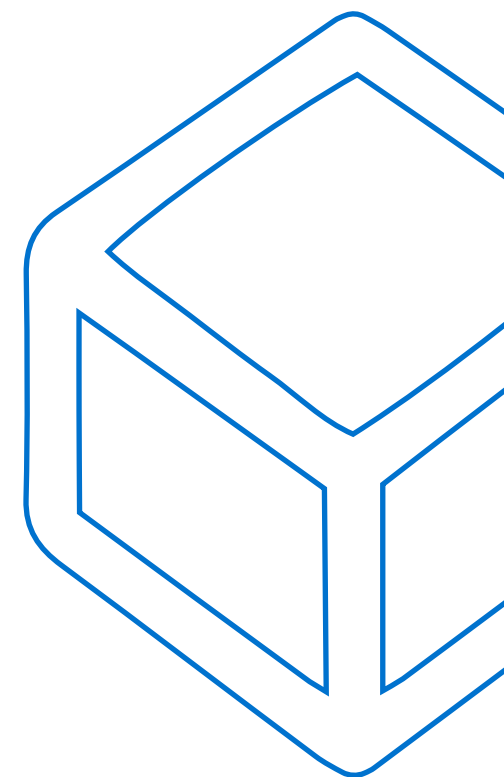
Most process automation tools rely on a central database to store the status of running processes (a technical challenge known as state persistence). A single database creates a performance bottleneck that precludes high-volume use cases such as instant global payments or trade settlement, and scaling by adding engines and databases costs computing resources, complicates operations, and fragments process management and reporting.

Monolithic Automation Platforms

Monolithic automation platforms are sold as all-in-one solutions for designing, executing, managing, and analyzing business processes, plus adjacent capabilities such as process discovery, low-code UI creation, and connectors for popular enterprise tools. Their defining quality is architectural: they're built as monoliths, so organizations must implement the whole platform instead of choosing the components that best fit each use case.

There are three main categories:

- **Traditional monolithic automation platforms** originated as business process management suites (BPMS), with process design and management features, decision automation, and form-building tools.
- **Modern monolithic automation platforms** emerged as vendors added process automation to products originally built for customer relationship management (CRM), IT service management (ITSM), or personal productivity. Most of these suites now also ship agent builders, so teams can create AI agents that work on the suite's own data and act within the suite's own processes.
- **Low-code application development platforms (LCAPs)** let people who aren't professional developers build applications, which often execute a limited number of tasks within a larger end-to-end process.



Challenges

Agents deepen the automation island

A monolithic platform automates well within its own boundaries and poorly beyond them, and the suites' new agent builders make this trap more acute. Every agent built inside a suite is designed for, governed by, priced by, and visible to that suite alone; it works best on the suite's data and struggles the moment a process crosses into another vendor's territory, which is exactly what core business processes do. Most organizations run several suites at once, and as agent adoption grows in each, coordination across platforms becomes harder: agents duplicate work, make conflicting decisions, and create compliance blind spots no single suite can see.

End-to-end processes almost always span multiple systems of record. A neutral orchestration layer can coordinate the agents, people, and workflows of every suite into one governed process; a monolith, by definition, cannot.

High development effort and no standards-based collaboration

Complex processes almost always require software development, and developers are most productive with the languages and tools they already know. Most automation monoliths instead require proprietary tools, so developers spend their time learning and maintaining vendor-specific skills, and connectors tend to be built in proprietary ways that are hard to customize or extend.

Process visualization suffers from the same problem: proprietary drag-and-drop formats leave little transparency for the developers who make processes executable, and processes can't be exported from one tool into another. Open standards such as Business Process Model and Notation (BPMN) and Decision Model and Notation (DMN) are fully transparent and portable without sacrificing user-friendly visualization.

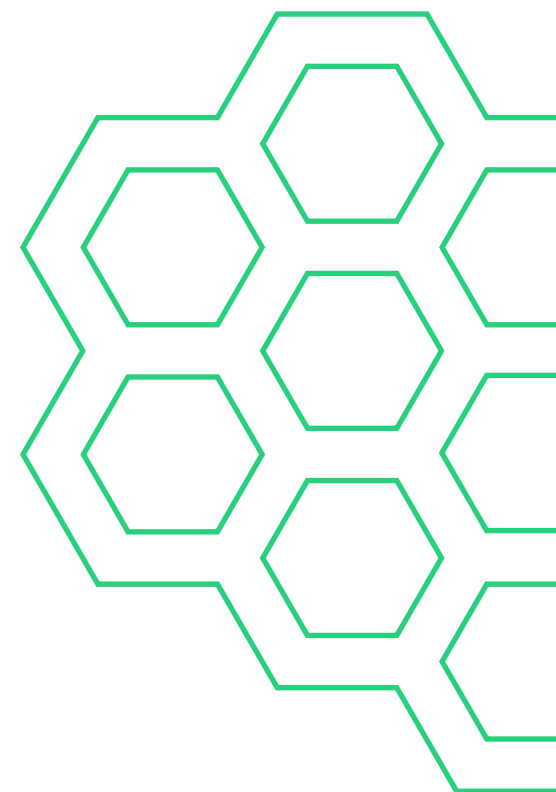
High total cost of ownership and lock-in

Automation monoliths lock organizations into a proprietary design and execution environment that requires extensive specialist consulting, both up front and every time processes change. The lock-in now extends to AI: when process logic, agents, and data are all bound to one vendor's ecosystem, every change in models, agent behavior, or architecture becomes rework, and switching costs grow with every automation built. The processes that break monolithic platforms aren't edge cases; they're the core ones, spanning dozens of endpoints with long-running logic, parallel branches, and escalation paths.

Microservices Orchestrators

Microservices are fine-grained, autonomous, loosely coupled services organized around business capabilities. Although they operate independently, there's usually an implied business process that requires each service to be invoked at the right time and under the right conditions. Microservices orchestrators coordinate that work, providing a process execution layer on top of microservices so engineering teams can understand, troubleshoot, and modify service interactions without compromising microservices design principles.

These tools are genuinely strong at what they were built for: durable, high-throughput coordination of services with dynamic scaling, in environments where the work is owned entirely by software development or IT teams rather than the business. Notably, some task automation vendors have recently launched orchestration products built on OEM-embedded microservices orchestrators; these inherit both the strengths and the limitations described here, while adding a dependency on an engine the vendor doesn't own.



Challenges

Processes live in code, so business visibility never arrives

It's rare for a business process to consist of microservices alone. Most IT landscapes also contain monolithic applications, legacy systems, user-facing front ends, and human work, and complete visibility requires all of it to be represented in a single process flow.

Orchestration logic lives in code or configuration files, so there's no graphical process model that business users can view, monitor, or help improve; any visualization reflects code and can't change without editing code. Business stakeholders are excluded from the process lifecycle, and IT must troubleshoot every issue. When something fails at 2 a.m., operations teams read logs instead of seeing the state of every in-flight case.

Agent governance requires custom instrumentation

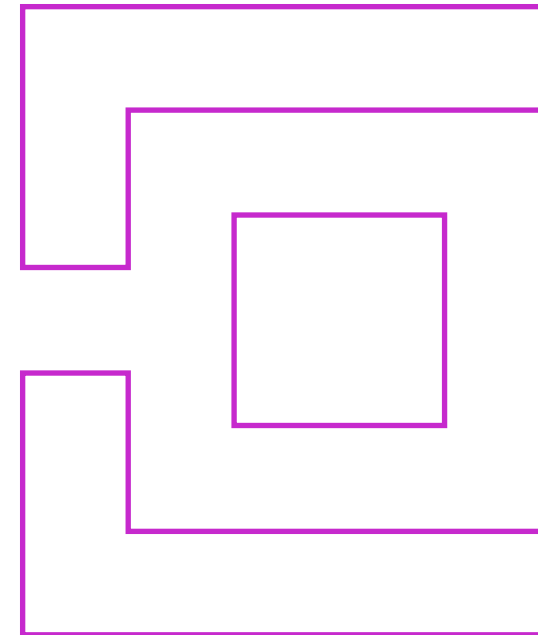
Code-first orchestrators can invoke agents just as they invoke any other service, but they were never designed to govern them. There is no shared model where business and compliance stakeholders can see what an agent is allowed to do, no built-in mechanism to enforce approval steps between an agent's decision and its action, and no process-level audit trail of agent behavior. Every guardrail becomes custom instrumentation, and that technical debt grows with every agent deployed.

Task and Integration Automation Tools

Task automation is the use of software to perform individual tasks without human intervention. Integration automation moves, copies, synchronizes, and transforms data as it flows between systems. Both are necessary: an end-to-end process is made up of many tasks that constantly exchange data, and combined with process orchestration they achieve process automation. The tools in this category are essential parts of that equation. But they are parts, not the whole.

- **RPA tools** use screen-scraping to simulate mouse and keyboard actions, automating tasks within legacy systems that lack an API.
- **Point solutions** automate a specific type of task; for example, intelligent document processing (IDP) tools read data from PDFs, and workload automation tools provision infrastructure.
- **Integration platforms as a service (iPaaS)** act as “glue” between systems, offering pre-built connectors for a wide variety of products along with point-to-point data transformation and mapping.
- **Workflow automation tools** let technical and semi-technical builders visually chain tasks, connectors, and LLM calls; developer-friendly and fast to prototype with, they’ve become a popular entry point for AI-assisted automation experiments.

Typical use cases include mainframe and legacy system interaction, document-driven work such as invoice data extraction, synchronizing data between enterprise resource planning (ERP) and CRM systems, and chaining notifications, data lookups, and simple LLM-powered steps.



Challenges

The process isn't in the tool

These tools automate one piece of work at a time or move data from point to point, so the concept of “process” isn't built into them: the tool isn't aware of tasks already executed and can't decide what should execute next based on end-to-end state.

Many can trigger a next action, and teams use that to chain automated tasks together, increasingly with LLM calls in the chain. Chaining delivers quick wins, but it is not process orchestration. It doesn't make the process visible, it produces brittle exception handling, and it creates no shared operational picture. As chained automations multiply, organizations hit sprawl: dozens or hundreds of automations no one can see end to end, each failing in its own way. Task-level wins accumulate while process-level outcomes stall.

No support for long-running processes

Business-critical processes commonly run for hours, days, weeks, or longer. A mortgage takes weeks to complete, involves sensitive data, and is strictly regulated; tracking its status, with an audit trail, is non-negotiable. That tracking is a technical challenge known as state persistence, and tools in this category can't store the information or access it at the right moment, such as when the applicant signs closing paperwork weeks later. Without state persistence, they cannot implement long-running processes or provide the durable coordination AI agents need when they wait on people, systems, or one another.

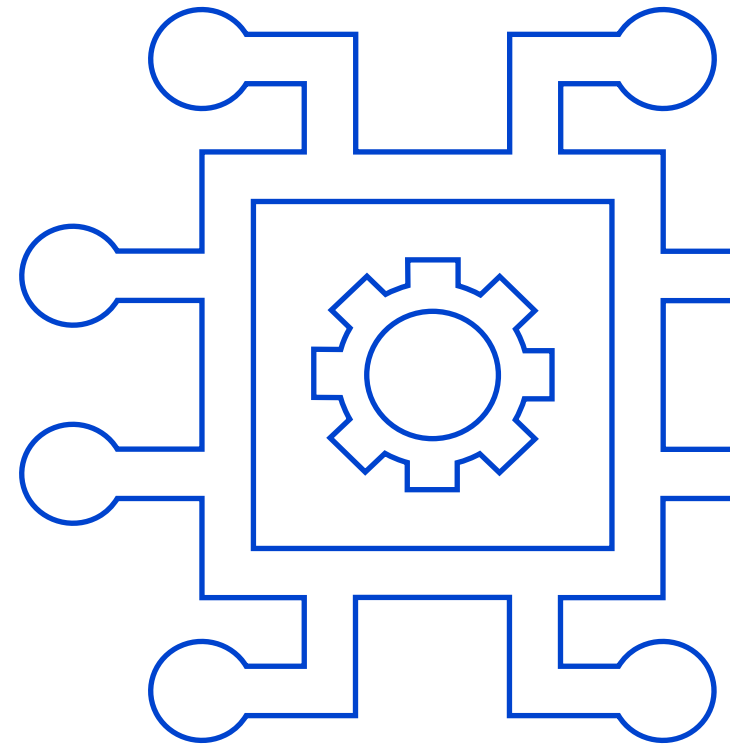
Costs that scale with consumption

Vendors in this category often charge based on resource consumption: connectors, automations, tasks executed, or data processed. Consumption-based pricing can suit small teams with predictable needs, but it doesn't scale well for large organizations or projects with unpredictable throughput, and the need to strictly control spend can prevent broad rollout, capping the value of the investment.

AI Agent Frameworks and Platforms

AI agent frameworks are developer libraries for building agents, handling agent creation, reasoning loops, tool calling, and memory; AI agent platforms add managed runtimes and visual builders for creating, deploying, and running agents. These tools have made it dramatically easier to build capable agents, and they're genuinely good at what they do.

This category is different from the others in this paper: it is complementary to process orchestration, not an alternative to it. Agent frameworks build agents; an orchestration layer governs how those agents participate in business processes. Open protocols such as Model Context Protocol (MCP) and Agent2Agent (A2A) let an orchestration layer coordinate agents from any framework or platform, so an investment in agent-building tools is preserved, not replaced, by orchestration.



Challenges

The prototype trap

The pattern is now familiar: an agent succeeds in a controlled pilot but cannot be safely scaled, governed, or integrated into production operations, leaving the organization stuck at proof of concept indefinitely. The gap is rarely the agent itself; it's everything around it. A production process needs consistent control over what agents may do, a complete audit trail of what they did, human escalation when confidence is low, and integration into the end-to-end journey the customer actually experiences. Agent frameworks don't provide any of this, because it was never their job.

No end-to-end process context or durable coordination

An agent built with a framework operates on the task in front of it. It has no view of the business process it participates in: what has already happened, what is waiting, what is allowed to happen next. An agent can process income documents brilliantly and still have no idea that the application has been waiting nine days on an appraisal.

Multi-agent scenarios raise the bar further. Real processes require agents to wait on one another, on systems, and on people for minutes, hours, or days; to correlate messages between agents that don't know about each other; and to handle timeouts, retries, and human escalation. That takes durable, stateful coordination that agent frameworks don't hold, and teams that build it themselves end up with glue code that grows more fragile as models and requirements evolve.

Governance is left to the builder

In an agent framework, guardrails are typically instructions in a prompt or checks written in code: conventions, not enforcement. There is no structural mechanism guaranteeing a required step happens between an agent's decision and its action, and no process-level record a regulator or auditor can follow. When something goes wrong at enterprise scale (as it will), per-agent logs scattered across deployments are not an answer to the question "why did this happen?"

None of this diminishes what these tools are for. But the answer to the prototype trap isn't a better agent. So what does it actually take to give agents real operational authority?

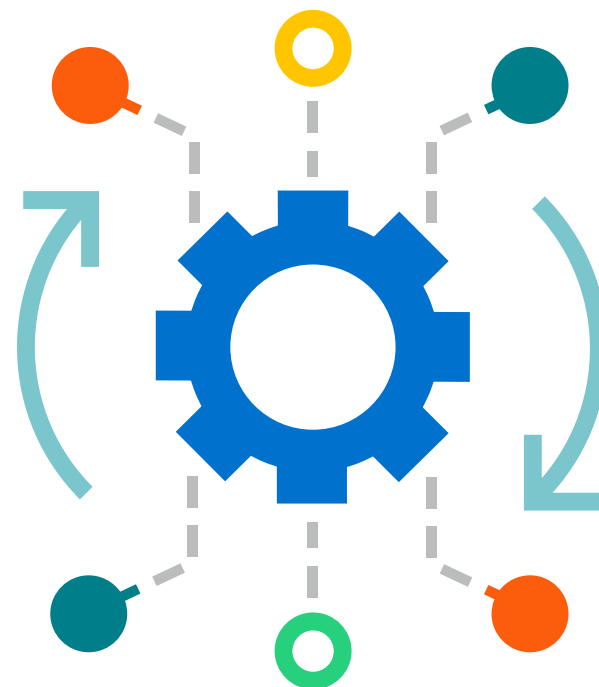
Agentic Orchestration with Camunda

Most organizations use hundreds or thousands of off-the-shelf and homegrown applications to execute core processes, and now they're adding AI agents from multiple vendors on top. A process spanning that many systems fragments into parts that execute in isolation, with a loss of visibility, integration, and control. Camunda eliminates this fragmentation by orchestrating every automated task, human task, and AI agent in an end-to-end business process, no matter how many applications, systems, or vendors are involved. Here is how Camunda delivers each quality used to evaluate the landscape.

End-to-end process orchestration

Camunda orchestrates both automated and manual tasks across complete business processes, supporting advanced workflow patterns such as parallel execution, automated escalations, message correlation, and exception handling. Teams can add any endpoint (APIs, microservices, RPA bots, IoT devices, AI agents) without restricting their process design choices or tools.

The results are measurable at the process level, not just the task level. [Harmoney](#), a digital lender processing 12,000 loan applications a month, orchestrates identity verification, income assessment, and credit decisioning as a single flow: decisions that took four days now take about 17 minutes, roughly 90 times faster.



Agentic orchestration, outside and inside the agent

Camunda is the only platform that orchestrates agents through both patterns in one model. Outer orchestration (outside the agent) coordinates agents as participants in a larger end-to-end process, alongside people and systems. Inner orchestration (inside the agent) injects enforceable steps between an agent's decision and its action, embedding must-follow operating procedures between the agent's brain (the LLM) and its hands (the tools). If an agent decides to email Maya that her closing date is at risk, the process can force a human approval step before that message goes out. For agents built in Camunda, this inner orchestration approach is covered by a US patent pending. This is what makes it safe to give agents real operational authority.

Real processes rarely involve just one agent. A specialist agent can be exposed as a tool to another agent, so an orchestrator agent delegates work to sub-agents and waits, for minutes or for days, without losing state. What those agents do not do is talk to each other. Coordination runs through the process rather than through negotiation between peers, which means each agent receives only the context its step requires, the state lives in one place, and there's one record to inspect instead of a transcript of inter-agent messages to reconstruct after the fact.

Unlike platforms that bolt AI onto processes as isolated tasks, Camunda treats agents as first-class citizens: modeled in BPMN like every other part of the process, executed by the same workflow engine, and visible at every stage, with autonomy dialed up or down per step. Straight-through processing handles the predictable majority; agent reasoning handles exceptions and complex decisions; humans step in where judgment or accountability demands it. [SüdLeasing](#) combines this blend with intelligent document processing to reach an 80–90% automation rate for document processing.

Support for long-running processes

Camunda process orchestration is based on the BPMN standard, so processes are both graphical and executable, with built-in support for long-running processes: state persistence, data correlation, and timeouts are handled automatically, whether a process runs for seconds or months.

Agent memory is process state: held and persisted by the process rather than inside the agent, inspectable like any other process data, and backed by whichever store you choose, from the process instance itself to an external vector database.

The same durability powers multi-agent coordination: orchestrator agents can kick off work to other agents and wait for minutes, hours, or days without losing state, while Camunda correlates messages between agents that don't know about each other and manages simultaneous waits with built-in timeout, retry, and human escalation. Teams can also deploy new process versions while current ones are running and migrate in-flight work at runtime, critical for live operations that can't stop.

Observability, auditability, and optimization

Camunda is the operational control room for AI-powered processes. Observability isn't bolted on: every process execution, agent decision, and exception is captured automatically, creating a complete, queryable process audit trail without additional instrumentation.

Camunda Operate provides real-time visibility and intervention so teams can monitor work in flight and safely resolve issues in running instances, while Camunda Optimize delivers continuous performance insights, surfacing bottlenecks, tracking service-level agreement (SLA) adherence, and identifying where automation needs improvement. When a regulator

asks why something happened, when operations leadership asks where a case is, or when an agent behaves unexpectedly, the answer and the full trace are there.

Agent behavior is process data like any other: Optimize reports which tools an agent called, how often, and how many tokens an instance consumed, with thresholds set against a budget so cost overruns surface early.

One shared process model

Camunda uses the globally recognized BPMN and DMN standards to bridge the gap between IT and the business. The process model that business and IT stakeholders design together is the same model the workflow engine executes: the documentation and the execution are one artifact, so nothing is lost in translation between requirements and implementation. Monitoring overlays status and incidents on that same model, and reports show historical performance with the same visualization, so business users stay involved across the entire lifecycle instead of being limited to design time.

Open and composable architecture

Camunda offers loosely coupled components that integrate fully with one another yet fit into an existing technical architecture. Composability is a genuine tradeoff, worth stating plainly: an orchestration-first platform asks more of your team up front than an all-in-one suite does. There is no walled garden to switch on; teams compose the solution from Camunda's components, their own stack, and the endpoints they choose. In return you get everything the walled gardens can't offer: architectural freedom, best-of-breed choices at every layer, and no lock-in.

Components can be deployed on premises, in public or private clouds, in hybrid configurations, or consumed as SaaS, and developers work in their preferred editor and language, keep code in version control, and manage containerized applications on platforms such as Kubernetes. There's no Camunda-specific way of working to adopt.

Openness extends to AI. Camunda orchestrates agents from any provider (commercial, open source, or custom-built) via MCP and A2A protocol connectors, so organizations can change LLMs, agent runtimes, and decisioning approaches without rewriting process logic. And because Camunda is built on BPMN, an open ISO standard, process logic is portable: models built today remain valid and executable regardless of how the AI landscape evolves.

Highly scalable and resilient

Camunda's cloud-native workflow engine, Zeebe, is designed for high-throughput use cases out of the box. While traditional workflow engines rely on a central database that becomes a bottleneck, Zeebe distributes processing across nodes using event-streaming technology, so capacity grows by adding nodes and millions of concurrent process instances run at consistently low latency. Zeebe's brokers form a peer-to-peer network with no single point of failure; if a node or an entire datacenter goes down, another takes over automatically with no data loss.

This is scalability that customers measure in business terms: [Halkbank](#) improved its money transfer process time by 600%, from 54 seconds to nine seconds, with 50% fewer errors.

Low total cost of ownership

Many automation tools have a reputation for high total cost of ownership due to long ramp-up times, ongoing consulting fees, and rework whenever the business or the technology changes. Camunda keeps long-run costs down structurally rather than through promises:

- **No proprietary skills tax.** Open standards and support for developers' existing toolchains mean teams don't pay an ongoing tax of proprietary skills, certified specialists, and vendor consulting.
- **Composability instead of consulting dependency.** Teams adopt only the components they need and integrate them into the existing stack, rather than funding a platform migration.
- **Investment that survives change.** Process models are portable and vendor-neutral, so changes in AI models, agent runtimes, or infrastructure don't force expensive rework of process logic.

These structural qualities compound over time: [Atlassian](#) reports a 30x return on its automation spend with Camunda.





Conclusion

Every process running your business today was designed for a world without AI. That doesn't mean everything must be redesigned tomorrow. But it does mean the tools you choose now determine whether redesign is ever possible. Task and integration automation deliver necessary pieces; monolithic suites automate within their walls; microservices orchestrators serve engineering teams; agent frameworks build capable agents. What none of them provide is the orchestration layer that coordinates agents, people, and systems across end-to-end processes with the governance that production demands.

With agentic orchestration from Camunda, your organization can:

- **Deliver better customer experiences** by orchestrating every task, system, and agent in an end-to-end business process
- **Get to value and achieve business goals faster** with enforceable guardrails, human oversight, and a complete audit trail for every agent decision
- **Increase business agility** by continuously improving processes through standards-based collaboration between business and IT
- **Drive operational cost savings** by automating manual work, minimizing errors, and optimizing processes with deep insight into performance



CAMUNDA

See what comes next

This paper opened with an uncomfortable idea: every process currently running your business is legacy. It was designed before AI existed, before agents could reason and act, before the rules changed. Choosing an orchestration layer is how you make redesign possible, but the redesign itself is a bigger question than tooling.

At CamundaCon 2026, Camunda CEO Jakob Freund and CTO Daniel Meyer laid out what they call the Great Re-Engineering: why every organization will either reinvent how it operates around AI or watch a competitor do it first.

[Watch the opening keynote on demand.](#)

About Camunda

Camunda is the enterprise platform for agentic orchestration, enabling organizations to coordinate AI agents, people, and systems across complex, end-to-end business processes. With built-in governance, auditability, and human oversight, Camunda gives enterprises the control they need to move AI from pilots to production — safely and at scale. Trusted by over 700 organizations worldwide, including 9 of top 10 US banks, Camunda helps enterprises boost operational efficiency, accelerate time-to-value, and deliver better customer experiences.

Visit camunda.com.

